

```

package main

import (
    "fmt"

    "github.com/connerohnesorge/goffice/drawingml"
    "github.com/connerohnesorge/goffice/presentation"
)

// createAreaChartSlide creates a slide with an area chart showing cumulative
// revenue.
// The chart displays stacked revenue growth for three product lines.
func createAreaChartSlide(
    doc *presentation.Document,
) error {
    slidePart, err := doc.AddSlide()
    if err != nil {
        return fmt.Errorf(
            errFailedToAddSlide,
            err,
        )
    }
    slide := slidePart.Slide()

    // Add slide title
    addTitleToSlide(
        slide,
        "Cumulative Revenue Growth",
    )

    // Create chart with title
    chartSpace, chart := createChartWithTitle(
        "Revenue Growth by Product Line",
    )
    plotArea := chart.PlotArea()

    // Add stacked area chart
    areaChart := plotArea.AddAreaChart(
        drawingml.GroupingStacked,
    )

    // Add series for each product (note: seriesName not used for area charts)
    addAreaChartSeries(
        areaChart,
        &chartSeriesConfig{
            seriesName:    "",
            categoryRange: areaCategoryRange,
            categoryValues: areaCategories,
            valuesRange:    areaProductARange,
            values:         areaProductAData,
            seriesIndex:    0,
        },
    )
    addAreaChartSeries(
        areaChart,
        &chartSeriesConfig{
            seriesName:    "",
            categoryRange: areaCategoryRange,
            categoryValues: areaCategories,

```

```

                valuesRange:    areaProductBRange,
                values:         areaProductBData,
                seriesIndex:    1,
            },
        )
    addAreaChartSeries(
        areaChart,
        &chartSeriesConfig{
            seriesName:    "",
            categoryRange: areaCategoryRange,
            categoryValues: areaCategories,
            valuesRange:   areaProductCRange,
            values:        areaProductCData,
            seriesIndex:   2,
        },
    )

    // Configure axes
    areaChart.AddAxisID(categoryAxisID)
    areaChart.AddAxisID(valueAxisID)
    addChartAxes(plotArea)

    // Add legend at the top
    legend := drawingml.NewLegendWithPosition(
        drawingml.LegendPositionTop,
    )
    chart.SetLegend(legend)

    return addChartToSlide(slidePart, chartSpace)
}

// createScatterChartSlide creates a slide with a scatter chart showing
// correlation.
// The chart displays the relationship between price and demand for two product
// lines.
func createScatterChartSlide(
    doc *presentation.Document,
) error {
    slidePart, err := doc.AddSlide()
    if err != nil {
        return fmt.Errorf(
            errFailedToAddSlide,
            err,
        )
    }
    slide := slidePart.Slide()

    // Add slide title
    addTitleToSlide(
        slide,
        "Price vs Demand Analysis",
    )

    // Create chart with title
    chartSpace, chart := createChartWithTitle(
        "Product Demand vs Price Point",
    )
    plotArea := chart.PlotArea()

```

```

// Add scatter chart with lines and markers
scatterChart := plotArea.AddScatterChart(
    drawingml.ScatterStyleLineMarker,
)

// Add series for each product line
addScatterChartSeries(
    scatterChart,
    &scatterSeriesConfig{
        xRange:      scatterXRange,
        xValues:      scatterPrices,
        yRange:      scatterProduct1YRange,
        yValues:      scatterProduct1Demand,
        seriesIndex: 0,
    },
)
addScatterChartSeries(
    scatterChart,
    &scatterSeriesConfig{
        xRange:      scatterXRange,
        xValues:      scatterPrices,
        yRange:      scatterProduct2YRange,
        yValues:      scatterProduct2Demand,
        seriesIndex: 1,
    },
)

// Configure axes (both value axes for scatter charts)
scatterChart.AddAxisID(categoryAxisID)
scatterChart.AddAxisID(valueAxisID)
addValueAxes(plotArea)

// Add legend at the bottom
legend := drawingml.NewLegendWithPosition(
    drawingml.LegendPositionBottom,
)
chart.SetLegend(legend)

return addChartToSlide(slidePart, chartSpace)
}

```